4 Solution Concepts for Games

What does it mean for agents to interact optimally in a multi-agent system? In other words, what is a *solution* to a game? This is a central question of game theory, and many different *solution concepts* have been proposed that specify when a collection of agent policies constitutes a stable or desirable outcome. While Chapter 3 introduced the basic game models to formalize multi-agent environments and interaction, this chapter will introduce a series of solution concepts. Together, a game model and solution concept define a learning problem in MARL (Figure 4.1).

For common-reward games, in which all agents receive the same reward, a straightforward definition for a solution is to maximize the expected return received by all agents (but finding such a solution may not be simple at all, as we will see in later chapters). However, if the agents have differing rewards, the definition of a solution concept becomes more complicated.

In general, a solution to a game is a *joint policy* that consists of one policy for each agent and satisfies certain properties. These properties are expressed in terms of the expected returns yielded to each agent under the joint policy and the relations between the agents' returns. Thus, this chapter will begin by giving a universal definition of expected returns that applies to all of the game models introduced in Chapter 3. Using this definition, we will introduce a hierarchy of increasingly general equilibrium solution concepts that include the minimax equilibrium, Nash equilibrium, and correlated equilibrium. We will also discuss solution refinements such as Pareto-optimality and social welfare and fairness, and alternative solution concepts such as no-regret. We finish the chapter with a discussion of the computational complexity of computing Nash equilibria.

Note that our definitions of solution concepts and their stated existence properties assume *finite* game models, as defined in Chapter 3. In particular, we assume finite state, action, and observation spaces, and a finite number of agents. Games with infinite elements, such as continuous actions and observations, can

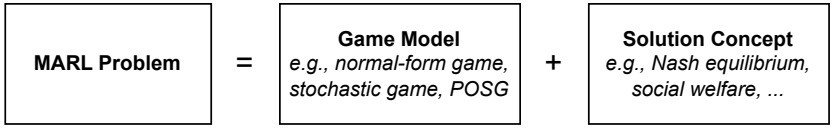


Figure 4.1: A MARL problem is defined by the combination of a game model, which defines the mechanics of the multi-agent system and interactions, and a solution concept, which specifies the desired properties of the joint policy to be learned. (See also Figure 2.1, page 20.)

use analogous definitions (e.g., by using densities and integrals) but may have different existence properties for solutions.[1]

Game theory has produced a wealth of knowledge on foundational questions about solution concepts, including: For a given type of game model and solution concept, is a solution guaranteed to exist in the game? Is the solution unique (i.e., only one solution exists) or are there potentially many solutions, even infinitely many? And is a given learning and decision rule, when used by all agents, guaranteed to converge to a solution? For a more in-depth treatment of such questions, we recommend the books of Fudenberg and Levine (1998), Young (2004), Shoham and Leyton-Brown (2008), and Owen (2013).

4.1 Joint Policy and Expected Return

A solution to a game is a *joint policy*, $\pi = (\pi_1, ..., \pi_n)$, which satisfies certain requirements defined by the solution concept in terms of the expected return, $U_i(\pi)$, yielded to each agent i under the joint policy. We seek a universal definition of the expected return $U_i(\pi)$, which applies to all of the game models introduced in Chapter 3, so that our definitions of solution concepts also apply to all game models. Thus, we will define the expected return in the context of the POSG model (Section 3.4), which is the most general game model used in this book and includes both stochastic games and normal-form games.

We start by introducing some additional notation. In a POSG, let $\hat{h}^t = \{(s^\tau, o^\tau, a^\tau)_{\tau=0}^{t-1}, s^t, o^t\}$ denote a *full history* up to time t, consisting of the states, joint observations, and joint actions of all agents in each time step before t, and the state s^t and joint observation o^t at time t. The function $\sigma(\hat{h}^t) = (o^0, ..., o^t)$

1. For example, every two-agent zero-sum normal-form game with finite action spaces has a unique minimax game value, but zero-sum games with continuous actions exist that do not have a minimax game value (Sion and Wolfe 1957). Sufficient conditions for equilibrium existence in continuous games have been studied in works such as Debreu (1952), Fan (1952), Glicksberg (1952), and Dasgupta and Maskin (1986).

returns the history of joint observations from the full history $\hat{h}^t$, and we sometimes abbreviate $\sigma(\hat{h}^t)$ to h^t if it is clear from context. We define the probability of a joint observation, $\mathcal{O}(o^t \mid a^{t-1}, s^t)$, as the product $\prod_{i \in I} \mathcal{O}_i(o_i^t \mid a^{t-1}, s^t)$.

To make our definitions valid for both finite and infinite time steps, we use discounted returns[2] and assume the standard convention of absorbing states, which we defined in Section 2.3. That is, once an absorbing state has been reached, the game will subsequently always transition into that same state with probability 1 and give a reward of 0 to all agents. To simplify our definitions below, we also assume that the observation functions and policies of all agents become deterministic (i.e., assign probability 1 to a certain observation and action, respectively) once an absorbing state has been reached. Thus, our definitions encompass infinite episodes as well as episodes that terminate after a finite number of time steps, including non-repeated normal-form games that always terminate after a single time step.[3]

In the following, we provide two equivalent definitions of expected returns. The first definition is based on enumerating all full histories in the game, while the second definition is based on a Bellman-style recursion of value computations. These definitions are equivalent, but can provide different perspectives and have been used in different ways. In particular, the first definition resembles a linear sum and may be easier to interpret, while the second definition uses a recursion that can be operationalized such as in value iteration for games (introduced in Section 6.1).

History-based expected return: Given a joint policy π, we can define the expected return for agent i under π by enumerating all possible full histories and summing the returns for agent i in each history, weighted by the probability of generating the history under the POSG and joint policy π. Formally, define the set $\hat{H}$ to contain all full histories $\hat{h}^t$ for $t \to \infty$.[4] Then, the expected return for agent i under joint policy π is given by

$$U_i(\pi) = \lim_{t \to \infty} \mathbb{E}_{\hat{h}^t \sim (\mu, \mathcal{T}, \mathcal{O}, \pi)} \left[u_i(\hat{h}^t) \right] \tag{4.1}$$

$$= \sum_{\hat{h}^t \in \hat{H}} \Pr(\hat{h}^t \mid \pi)\, u_i(\hat{h}^t) \tag{4.2}$$

2. Alternative definitions based on average rewards instead of discounted returns are also possible for games (Shoham and Leyton-Brown 2008).

3. In the context of non-repeated normal-form games, the expected return for an agent simply becomes the expected reward of that agent, so we use the term "expected return" in all game models including non-repeated normal-form games.

4. Recall from Chapter 3 that we assume finite game models, and that a^{t-1} before time $t = 0$ is the empty set.

where $\Pr(\hat{h}^t \mid \pi)$ is the probability of full history $\hat{h}^t$ under π,

$$\Pr(\hat{h}^t \mid \pi) = \mu(s^0)\mathcal{O}(o^0 \mid \emptyset, s^0) \prod_{\tau=0}^{t-1} \pi(a^\tau \mid h^\tau)\mathcal{T}(s^{\tau+1} \mid s^\tau, a^\tau)\mathcal{O}(o^{\tau+1} \mid a^\tau, s^{\tau+1}) \tag{4.3}$$

and $u_i(\hat{h}^t)$ is the discounted return for agent i in $\hat{h}^t$,

$$u_i(\hat{h}^t) = \sum_{\tau=0}^{t-1} \gamma^\tau \mathcal{R}_i(s^\tau, a^\tau, s^{\tau+1}) \tag{4.4}$$

with discount factor $\gamma \in [0, 1]$.

We use $\pi(a^\tau \mid h^\tau)$ to denote the probability of joint action a^τ under joint policy π after joint-observation history h^τ. If we assume that agents act independently, then we can define

$$\pi(a^\tau \mid h^\tau) = \prod_{j \in I} \pi_j(a_j^\tau \mid h_j^\tau). \tag{4.5}$$

If agents do not act independently, such as in correlated equilibrium (Section 4.6) and methods such as central learning (Section 5.3.1), then $\pi(a^\tau \mid h^\tau)$ can be defined accordingly.

Recursive expected return: Analogous to the Bellman recursion used in MDP theory (Section 2.4), we can define the expected return for agent i under joint policy π via two interlocked functions V_i^π and Q_i^π, defined below. In the following, we use $s(\hat{h})$ to denote the last state in $\hat{h}$ (i.e., $s(\hat{h}^t) = s^t$), and we use $\langle\rangle$ to denote the concatenation operation.

$$V_i^\pi(\hat{h}) = \sum_{a \in A} \pi(a \mid \sigma(\hat{h}))Q_i^\pi(\hat{h}, a) \tag{4.6}$$

$$Q_i^\pi(\hat{h}, a) = \sum_{s' \in S} \mathcal{T}(s' \mid s(\hat{h}), a) \left[\mathcal{R}_i(s(\hat{h}), a, s') + \gamma \sum_{o' \in O} \mathcal{O}(o' \mid a, s')V_i^\pi(\langle \hat{h}, a, s', o' \rangle) \right] \tag{4.7}$$

We can understand $V_i^\pi(\hat{h})$ as the expected return, also called *value*, for agent i when agents follow the joint policy π after the full history $\hat{h}$. Similarly, $Q_i^\pi(\hat{h}, a)$ is the expected return for agent i when agents execute the joint action a after $\hat{h}$ and then follow π subsequently. Given Equations 4.6 and 4.7, we can define the expected return for agent i from the initial state of the game as

$$U_i(\pi) = \mathbb{E}_{s^0 \sim \mu, o^0 \sim \mathcal{O}(\cdot \mid \emptyset, s^0)} \left[V_i^\pi(\langle s^0, o^0 \rangle) \right]. \tag{4.8}$$

The equivalence of the history-based and recursive definitions for U_i can, intuitively, be seen by viewing the former as enumerating all possible infinite full histories, while the latter recursively constructs an infinite tree rooted in the initial state s^0 (or, one tree for each possible initial state $s^0 \sim \mu$) in which the

branching corresponds to the different possible full histories. In this chapter, we will define most of the presented solution concepts based on U_i, with the understanding that U_i can be defined via the two equivalent definitions given above.

4.2 Best Response

Many existing solution concepts, including most of the solution concepts introduced in this chapter, can be expressed compactly based on *best responses*. Given a set of policies for all agents other than agent i, denoted by $\pi_{-i} = (\pi_1, ..., \pi_{i-1}, \pi_{i+1}, ..., \pi_n)$, a best response for agent i to π_{-i} is a policy π_i that maximizes the expected return for i when played against π_{-i}. Formally, the set of best-response policies for agent i is defined as

$$\text{BR}_i(\pi_{-i}) = \arg\max_{\pi_i} U_i(\langle \pi_i, \pi_{-i} \rangle) \tag{4.9}$$

where $\langle \pi_i, \pi_{-i} \rangle$ denotes the complete joint policy consisting of π_i and π_{-i}. For convenience and to keep our notation lean, we will sometimes drop the $\langle\rangle$ (e.g., $U_i(\pi_i, \pi_{-i})$).

Note that the best response to a given π_{-i} may not be unique, meaning that $\text{BR}_i(\pi_{-i})$ may contain more than one best-response policy. For example, in a non-repeated normal-form game, there may be multiple actions for agent i that achieve equal maximum expected return against a given π_{-i}, in which case any probability assignment to these actions is also a best response. (We will see a concrete example in the Rock-Paper-Scissors game in Section 4.3.)

Besides being useful for compact definitions of solution concepts, best response operators have also been used in game theory and MARL to iteratively compute solutions. Two example methods we will see include fictitious play (Section 6.3.1) and joint-action learning with agent modeling (Section 6.3.2), among others.

4.3 Minimax

Minimax is a solution concept defined for two-agent zero-sum games, in which one agent's reward is the negative of the other agent's reward (Section 3.1). A classical example of such games is the matrix game Rock-Paper-Scissors (see Figure 3.2(a), page 46). More complex examples with sequential moves include games such as chess and Go. The existence of minimax solutions for normal-form games was first proven in the foundational game theory work of von Neumann (1928) (see also von Neumann and Morgenstern (1944)).

Definition 5 (Minimax solution) *In a zero-sum game with two agents, a joint policy* $\pi = (\pi_i, \pi_j)$ *is a* minimax solution *if*

$$U_i(\pi) = \max_{\pi_i'} \min_{\pi_j'} U_i(\pi_i', \pi_j') \tag{4.10}$$

$$= \min_{\pi_j'} \max_{\pi_i'} U_i(\pi_i', \pi_j') \tag{4.11}$$

$$= -U_j(\pi).$$

Every two-agent zero-sum normal-form game has a minimax solution (von Neumann and Morgenstern 1944). Minimax solutions also exist in every two-agent zero-sum stochastic game with finite episode length, and two-agent zero-sum stochastic games with infinite episode length using discounted returns (Shapley 1953). Moreover, while more than one minimax solution may exist in a game, all minimax solutions yield the same unique value $U_i(\pi)$ for agent i (and, thus, agent j). This value is also referred to as the *(minimax) value of the game*.[5] Minimax values can also be defined for zero-sum games with more than two agents (see, for example, Equation 4.17 in Section 4.4).

In a minimax solution, each agent uses a policy that is optimized against a worst-case opponent that seeks to minimize the agent's return. Formally, there are two parts in the definition of minimax. Equation 4.10 is the minimum expected return that agent i can guarantee against *any* opponent. Here, π_i is agent i's *maxmin* policy and $U_i(\pi)$ is i's maxmin value. Conversely, Equation 4.11 is the minimum expected return that agent j can force on agent i. Here, we refer to j's policy π_j as the *minmax* policy against agent i, and $U_i(\pi)$ is agent i's minmax value. In a minimax solution, agent i's maxmin value is equal to its minmax value. Another way to interpret this is that the order of the min/max operators does not matter: agent i first announcing its policy followed by agent j selecting its policy, is equivalent to agent j first announcing its policy followed by agent i selecting its policy. Neither agent gains from such policy announcements.

A minimax solution can be understood more intuitively as each agent using a best-response policy against the other agent's policy. That is, (π_i, π_j) is a minimax solution if $\pi_i \in \text{BR}_i(\pi_j)$ *and* $\pi_j \in \text{BR}_j(\pi_i)$. In the non-repeated Rock-Paper-Scissors game, there exists a unique minimax solution which is for both agents to choose actions uniformly randomly (i.e., assign equal probability to all actions). This solution gives an expected return of 0 to both agents. In

5. Recall that we assume finite game models. For zero-sum games with continuous actions, examples exist that do not have a minimax game value (Sion and Wolfe 1957).

fact, it can be verified[6] in this game that if any agent i uses a uniform policy π_i, then *any* policy π_j for the other agent j is a best-response policy to π_i, and all best-response policies $\pi_j \in \text{BR}_j(\pi_i)$ yield an expected reward of 0 to agent j (and to agent i). This example shows that best responses need not be unique, and there can be many (even infinitely many) possible best-response policies. However, the joint policy (π_i, π_j) in which both policies are uniform policies is the *only* joint policy in which *both* policies are best responses to each other, making it a minimax solution. In Section 4.4, we will see that this best-response relation can also be applied to the more general class of general-sum games.

4.3.1 Minimax Solution via Linear Programming

For non-repeated zero-sum normal-form games, we can obtain a minimax solution by solving two linear programs, one for each agent. Each linear program computes a policy for one agent by minimizing the expected return of the other agent. Thus, agent i minimizes the expected return of agent j, and vice versa. We provide the linear program to compute the policy π_i for agent i; agent j's policy is obtained by constructing a similar linear program in which the indices i and j are swapped. The linear program contains variables x_{a_i} for each action $a_i \in A_i$ to represent the probability of selecting action a_i (thus, $\pi_i(a_i) = x_{a_i}$ defines the policy of agent i), as well as a variable U_j^* to represent the expected return of agent j, which is minimized as follows:

$$\text{minimize} \quad U_j^* \tag{4.12}$$

$$\text{subject to} \quad \sum_{a_i \in A_i} \mathcal{R}_j(a_i, a_j)\, x_{a_i} \;\leq\; U_j^* \qquad \forall a_j \in A_j \tag{4.13}$$

$$x_{a_i} \geq 0 \qquad \forall a_i \in A_i \tag{4.14}$$

$$\sum_{a_i \in A_i} x_{a_i} = 1 \tag{4.15}$$

In this linear program, the constraints in Equation 4.13 mean that no single action of agent j will achieve an expected return greater than U_j^* against the policy specified by $\pi_i(a_i) = x_{a_i}$. This implies that no probability distribution over agent j's actions will achieve a higher expected return than U_j^*. Finally, the constraints in Equations 4.14 and 4.15 ensure that the values of x_{a_i} form a

6. Assume agent 1's policy assigns probability $\frac{1}{3}$ to each action R, P, S. The expected reward to agent 2 for any policy π_2 is: $\pi_2(R)(\frac{1}{3}\mathcal{R}_2(R,R) + \frac{1}{3}\mathcal{R}_2(P,R) + \frac{1}{3}\mathcal{R}_2(S,R)) + \pi_2(P)(\frac{1}{3}\mathcal{R}_2(R,P) + \frac{1}{3}\mathcal{R}_2(P,P) + \frac{1}{3}\mathcal{R}_2(S,P)) + \pi_2(S)(\frac{1}{3}\mathcal{R}_2(R,S) + \frac{1}{3}\mathcal{R}_2(P,S) + \frac{1}{3}\mathcal{R}_2(S,S)) = \pi_2(R)(-\frac{1}{3} + \frac{1}{3}) + \pi_2(P)(+\frac{1}{3} - \frac{1}{3}) + \pi_2(S)(-\frac{1}{3} + \frac{1}{3}) = 0$. The same can be shown when switching the roles of agents 1 and 2.

valid probability distribution. Note that in a minimax solution, we will have $U_i^* = -U_j^*$ (see Definition 5).

Linear programs can be solved using well-known algorithms, such as the simplex algorithm, which runs in exponential time in the worst case but often is very efficient in practice; or interior-point algorithms, which are provably polynomial-time.

4.4 Nash Equilibrium

The *Nash equilibrium* solution concept applies the idea of a mutual best response to general-sum games with two or more agents. That such a solution exists in any general-sum non-repeated normal-form game was first proven in the celebrated work of Nash (1950).

Definition 6 (Nash equilibrium) *In a general-sum game with n agents, a joint policy* $\pi = (\pi_1, ..., \pi_n)$ *is a* Nash equilibrium *if*

$$\forall i, \pi_i' : U_i(\pi_i', \pi_{-i}) \leq U_i(\pi). \tag{4.16}$$

In a Nash equilibrium, no agent i can improve its expected return by changing its policy π_i specified in the equilibrium joint policy π, assuming the policies of the other agents stay fixed. This means that each agent's policy in the Nash equilibrium is a best response to the policies of the other agents, that is, $\pi_i \in \text{BR}_i(\pi_{-i})$ for all $i \in I$. Thus, Nash equilibrium generalizes minimax in that, in two-agent zero-sum games, the set of minimax solutions coincides with the set of Nash equilibria (Owen 2013). Nash (1950) first proved that every finite normal-form game has at least one Nash equilibrium.

Recall the matrix games shown in Figure 3.2 (page 46). In the non-repeated Prisoner's Dilemma matrix game, the only Nash equilibrium is for both agents to choose D. It can be seen that no agent can unilaterally deviate from this choice to increase its expected return. In the non-repeated coordination game, there exist three different Nash equilibria: (1) both agents choose action A, (2) both agents choose action B, and (3) both agents assign probability 0.5 to each action. Again it can be checked that no agent can increase its expected return by deviating from its policy in each equilibrium. Lastly, in the non-repeated Rock-Paper-Scissors game, the only Nash equilibrium is for both agents to choose actions uniform-randomly, which is the minimax solution of the game.

The previous examples illustrate two important aspects of Nash equilibrium as a solution concept. First, a Nash equilibrium can be *deterministic* in that each policy π_i in the equilibrium π is deterministic (i.e., $\pi_i(a_i) = 1$ for some $a_i \in A_i$), such as in Prisoner's Dilemma. However, in general, a Nash equilibrium may

be *probabilistic* in that the policies in the equilibrium use randomization (i.e., $\pi_i(a_i) < 1$ for some $a_i \in A_i$), such as in the coordination game and Rock-Paper-Scissors. In fact, some games only have probabilistic Nash equilibria but no deterministic Nash equilibria, such as the Rock-Paper-Scissors game. As we will see in Chapter 6, this distinction is important in MARL because some algorithms are unable to represent probabilistic policies, and hence cannot learn probabilistic equilibria. In the game theory literature, deterministic and probabilistic equilibria are also called "pure equilibria" and "mixed equilibria," respectively; see also Section 3.7 for terminology.

Second, a game may have multiple Nash equilibria, and each equilibrium may entail different expected returns for the agents. In the coordination game, the two deterministic equilibria give an expected return of 10 to each agent, while the probabilistic equilibrium gives an expected return of 5 to each agent. The Chicken game (Section 4.6) and Stag Hunt game (Section 5.4.2) are other examples of games with multiple Nash equilibria that each give different expected returns to the agents. This leads to the important question of which equilibrium the agents should converge to during learning and how this may be achieved. We will discuss this *equilibrium selection* problem in more depth in Section 5.4.2.

The existence of Nash equilibria has also been shown for stochastic games (Fink 1964; Filar and Vrieze 2012). In fact, for games with (infinite) sequential moves, there are various "folk theorems"[7] which essentially state that any set of feasible and enforceable expected returns $\hat{U} = (\hat{U}_1, ..., \hat{U}_n)$ can be achieved by an equilibrium solution if agents are sufficiently far-sighted (i.e., the discount factor γ is close to 1). The assumptions and details in different folk theorems vary and can be quite involved, and here we only provide a rudimentary description for intuition (for more specific definitions, see, e.g., Fudenberg and Levine 1998; Shoham and Leyton-Brown 2008). Broadly speaking, $\hat{U}$ is *feasible* if it can be realized within the game by some joint policy π; that is, there exists a π such that $U_i(\pi) = \hat{U}_i$ for all $i \in I$. And $\hat{U}$ is *enforceable* if each $\hat{U}_i$ is at least as large as agent i's minmax value[8]

$$v_i = \min_{\pi^{mm}_{-i}} \max_{\pi^{mm}_i} U_i(\pi^{mm}_i, \pi^{mm}_{-i}). \tag{4.17}$$

(Think: other agents $-i$ minimize the maximum achievable return for agent i.) Under these two conditions, we can construct an equilibrium solution that uses

7. The name "folk theorem" apparently came about because the general concept was known before it was formalized.

8. We first encountered minmax in Section 4.3 where it was defined for two agents, while in Equation 4.17 we define it for n agents (the *mm* superscript stands for minmax).

π to achieve $\hat{U}$, and if at any time t any agent i deviates from its policy π_i in π, the other agents will limit i's return to v_i by using their corresponding minmax policy π_{-i}^{mm} from Equation 4.17 indefinitely after t. Thus, since $v_i \leq \hat{U}_i$, i has no incentive to deviate from π_i, making π an equilibrium.

Given a joint policy π, how can one check whether it is a Nash equilibrium? Equation 4.16 suggests the following procedure, which reduces the multi-agent problem to n single-agent problems. For each agent i, keep the other policies π_{-i} fixed and compute an optimal best-response policy π_i' for i. If the optimal policy π_i' achieves a higher expected return than agent i's policy π_i in the joint policy π, that is, $U_i(\pi_i', \pi_{-i}) > U_i(\pi_i, \pi_{-i})$, then we know that π is not a Nash equilibrium. For non-repeated normal-form games, π_i' can be computed efficiently using a linear program (e.g., Albrecht and Ramamoorthy 2012). For games with sequential moves, π_i' may be computed using a suitable single-agent RL algorithm.

4.5 ϵ-Nash Equilibrium

The strict requirement of Nash equilibrium — that no agent can gain *anything* by unilaterally deviating from the equilibrium — can lead to practical issues when used in a computational system. It is known that for games with more than two agents, the action probabilities specified by the policies in the equilibrium may be irrational numbers (i.e., cannot be represented as a fraction of two integers). Nash himself pointed this out in his original work (Nash 1950). However, computer systems cannot fully represent irrational numbers using finite-precision floating-point approximations. Moreover, in many applications, reaching a strict equilibrium may be too computationally costly. Instead, it may be good enough to compute a solution that is sufficiently close to a strict equilibrium, meaning that agents could technically deviate to improve their returns but any such gains are sufficiently small.

The *ϵ-Nash equilibrium* relaxes the strict Nash equilibrium by requiring that no agent can improve its expected returns by more than some amount $\epsilon > 0$ when deviating from its policy in the equilibrium. Formally:

Definition 7 (ϵ-Nash equilibrium) *In a general-sum game with n agents, a joint policy $\pi = (\pi_1, ..., \pi_n)$ is an* ϵ-Nash equilibrium *for $\epsilon > 0$ if*

$$\forall i, \pi_i' : U_i(\pi_i', \pi_{-i}) - \epsilon \leq U_i(\pi). \tag{4.18}$$

Since the action probabilities specified by π_i' are continuous and we consider expected returns U_i, we know that every Nash equilibrium is surrounded by a region of ϵ-Nash equilibria for any $\epsilon > 0$. The exact Nash equilibrium corresponds

	C	D
A	100,100	0,0
B	1,2	1,1

Figure 4.2: Matrix game to illustrate that an ϵ-Nash equilibrium (B,D) ($\epsilon = 1$) may not be close to a real Nash equilibrium (A,C).

to $\epsilon = 0$. However, although it may be tempting to view ϵ-Nash equilibrium as an approximation of Nash equilibrium, it is important to note that an ϵ-Nash equilibrium may not be close to any real Nash equilibrium, in terms of the expected returns produced by the equilibrium. In fact, the expected returns under an ϵ-Nash equilibrium may be arbitrarily far away from those of any Nash equilibrium, even if the Nash equilibrium is unique.

Consider the example game shown in Figure 4.2. This game has a unique Nash equilibrium at (A,C).[9] It also has an ϵ-Nash equilibrium at (B,D) for $\epsilon = 1$, in which agent 2 could deviate to action C to increase its reward by ϵ. First, note that neither agent's return under the ϵ-Nash equilibrium is within ϵ of its return under the Nash equilibrium. Second, we can arbitrarily increase the rewards for (A,C) in the game without affecting the ϵ-Nash equilibrium (B,D). Thus, in this example, the ϵ-Nash equilibrium does not meaningfully approximate the unique Nash equilibrium of the game.

To check that a joint policy π constitutes an ϵ-Nash equilibrium for some given ϵ, we can use essentially the same procedure for checking Nash equilibrium described at the end of Section 4.4, except that we check for $U_i(\pi_i', \pi_{-i}) - \epsilon > U_i(\pi_i, \pi_{-i})$ to determine that π is not an ϵ-Nash equilibrium.

4.6 (Coarse) Correlated Equilibrium

A restriction of Nash equilibrium is that the agent policies must be probabilistically independent (as per Equation 4.5), which can limit the expected returns that can be achieved by the agents. *Correlated equilibrium* (Aumann 1974) generalizes Nash equilibrium by allowing for correlation between policies. In the general definition of correlated equilibrium, each agent i's policy is additionally conditioned on the outcomes of a private random variable d_i for the agent, which are governed by a joint probability distribution over $(d_1, ..., d_n)$ that is commonly known by all agents. Here, we will present a common version of correlated

9. Saying that a joint action a is an X-equilibrium (e.g., Nash) is a shorthand for saying that a deterministic joint policy π that assigns probability 1 to this joint action is an X-equilibrium.

equilibrium for non-repeated normal-form games, in which d_i corresponds to an action recommendation for agent i given by a joint policy π_c. At the end of this section, we will mention possible extensions to sequential-move games.

Definition 8 (Correlated equilibrium) *In a general-sum normal-form game with n agents, let $\pi_c(a)$ be a joint policy that assigns probabilities to joint actions $a \in A$. Then, π_c is a* correlated equilibrium *if for every agent $i \in I$ and every action modifier $\xi_i : A_i \to A_i$:*

$$\sum_{a \in A} \pi_c(a) \mathcal{R}_i(\langle \xi_i(a_i), a_{-i} \rangle) \leq \sum_{a \in A} \pi_c(a) \mathcal{R}_i(a) \tag{4.19}$$

Equation 4.19 states that in a correlated equilibrium, in which every agent knows the probability distribution $\pi_c(a)$ and its own recommended action a_i (but not the recommended actions for other agents), no agent can unilaterally deviate from its recommended actions in order to increase its expected return. Here, deviating from the recommended actions is represented by the action modifier ξ_i. An example of a correlated joint policy can be seen in the central learning approach discussed in Section 5.3.1, which trains a single policy π_c directly over the joint-action space $A_1 \times ... \times A_n$ and uses this policy to dictate actions to each agent.

It can be shown that the set of correlated equilibria contains the set of Nash equilibria (e.g., Osborne and Rubinstein 1994); namely, Nash equilibrium is a special case of correlated equilibrium in which the joint policy π_c is factored into independent agent policies $\pi_1, ..., \pi_n$ with $\pi_c(a) = \prod_{i \in I} \pi_i(a_i)$. In this case, since the agents' policies in a Nash equilibrium are independent, knowing one's own action a_i does not give any information about the action probabilities for the other agents $j \neq i$. Similarly to ϵ-Nash equilibrium defined in Equation 4.18, we may add $-\epsilon$ in the left part of Equation 4.19 to obtain ϵ-correlated equilibrium, in which no agent can unilaterally deviate from its recommended actions to increase its expected return by more than ϵ.

To see an example of how correlated equilibrium can achieve greater returns than Nash equilibrium, consider the Chicken matrix game shown in Figure 4.3. This game represents a situation in which two vehicles (agents) are on a collision course and can choose to either stay on course (S) or leave (L). In the non-repeated game, there are the following three Nash equilibria with associated expected returns for the two agents shown as pairs (U_i, U_j):

- $\pi_i(S) = 1,\ \pi_j(S) = 0 \ \to\ (7, 2)$
- $\pi_i(S) = 0,\ \pi_j(S) = 1 \ \to\ (2, 7)$
- $\pi_i(S) = \frac{1}{3},\ \pi_j(S) = \frac{1}{3} \ \to\ \approx (4.66, 4.66)$

	S	L
S	0,0	7,2
L	2,7	6,6

Figure 4.3: Chicken matrix game.

Now, consider the following joint policy π_c that uses correlated actions:

- $\pi_c(L,L) = \pi_c(S,L) = \pi_c(L,S) = \frac{1}{3}$
- $\pi_c(S,S) = 0$

The expected return under π_c to both agents is: $7 \cdot \frac{1}{3} + 2 \cdot \frac{1}{3} + 6 \cdot \frac{1}{3} = 5$. It can be verified that no agent has an incentive to unilaterally deviate from the action recommended to it by π_c, assuming knowledge of π_c (and without knowing the actions recommended to other agents). For example, say agent i received action recommendation L. Then, given π_c, i knows that agent j will choose S with probability 0.5 and L with probability 0.5. Thus, the expected return for i when choosing L is $2 \cdot \frac{1}{2} + 6 \cdot \frac{1}{2} = 4$, which is higher than the expected return for choosing S ($0 \cdot \frac{1}{2} + 7 \cdot \frac{1}{2} = 3.5$). Thus, i has no incentive to deviate from L.

The previous example also illustrates how a correlated equilibrium can again be described as mutual best responses between the agents. In correlated equilibrium, each action recommendation a_i given by π_c is a best response (i.e., achieves maximum expected return for i) to the conditional distribution $\pi_{-i}(a_{-i} \mid a_i) = \frac{\pi_c(\langle a_i, a_{-i} \rangle)}{\sum_{a'_i \in A_i} \pi_c(\langle a'_i, a_{-i} \rangle)}$ over actions a_{-i} recommended to the other agents by π_c given a_i.

We can obtain an even more general class of equilibrium solutions, called *coarse correlated equilibrium* (Moulin and Vial 1978),[10] by requiring that the inequality in Equation 4.19 only needs to hold for unconditional action modifiers, which satisfy $\forall a'_i \in A_i : \xi_i(a'_i) = a_i$ for some action a_i. In other words, each unconditional action modifier is just a constant action, with one ξ_i corresponding to each action $a_i \in A_i$. This solution concept means that each agent has to decide upfront, *before* seeing its recommended action, whether to follow the joint policy π_c assuming that the other agents follow it. If no agent can select a constant action (i.e., unconditional action modifier) to obtain a higher expected return compared to its expected return under π_c, as per Equation 4.19, then π_c is a coarse correlated equilibrium. Coarse correlated equilibria include correlated equilibria as a special case in which Equation 4.19 must hold for *all* possible action modifiers, not just unconditional action modifiers.

10. Moulin and Vial (1978) proposed this solution concept but did not name it in their work.

For sequential-move games, various definitions of correlated equilibrium exist (e.g., Forges 1986; Solan and Vieille 2002; von Stengel and Forges 2008; Farina, Bianchi, and Sandholm 2020). These definitions vary in a number of design choices and can be relatively complex. For instance, the private signals d_i may specify actions a_i that are revealed at each decision point, or they may specify entire policies π_i revealed once at the start of the game. The joint distribution over $d_1, ..., d_n$ and the action/policy modifier ξ_i may be conditioned on different types of information, such as the current game state, agent observation histories, or previous values of d_i (Solan and Vieille 2002). The sampled outcomes of d_i may or may not be revealed to agents, such as in coarse correlated equilibria, which assume that the outcomes of d_i are only revealed to agents if they "commit" to the equilibrium (e.g., Farina, Bianchi, and Sandholm 2020). Definitions of correlated equilibrium can also vary in how agents are treated if they deviate from the equilibrium; for example, no further action recommendations may be issued to an agent after it deviates from the action recommended by the equilibrium (von Stengel and Forges 2008).

4.6.1 Correlated Equilibrium via Linear Programming

For non-repeated normal-form games, we can compute a correlated equilibrium by solving a linear program. The linear program computes a joint policy π such that no agent can improve its expected return by deviating from the joint actions sampled from π. Thus, the linear program contains variables x_a for each joint action $a \in A$, to represent the probability of selecting a under the joint policy π (i.e., $\pi(a) = x_a$). To select between different possible equilibria, we here use an objective that maximizes the sum of the agents' expected returns (i.e., social welfare, see Section 4.9), but other objectives could be used such as maximizing expected returns for individual agents. This gives rise to the following linear program:

$$\text{maximize} \quad \sum_{a \in A} \sum_{i \in I} x_a \mathcal{R}_i(a) \tag{4.20}$$

$$\text{subject to} \quad \sum_{a \in A : a_i = a_i'} x_a \mathcal{R}_i(a) \geq \sum_{a \in A : a_i = a_i'} x_a \mathcal{R}_i(a_i'', a_{-i}) \quad \forall i \in I, a_i', a_i'' \in A_i \tag{4.21}$$

$$x_a \geq 0 \qquad \forall a \in A \tag{4.22}$$

$$\sum_{a \in A} x_a = 1 \tag{4.23}$$

The constraints in Equation 4.21 ensure the property that no agent can gain by deviating from the action a_i' sampled under the joint policy $\pi(a) = x_a$ to a different action a_i''. The constraints in Equations 4.22 and 4.23 ensure that the

values of x_a form a valid probability distribution. A similar linear program can be solved to compute a coarse correlated equilibrium, by replacing the constraints from Equation 4.21 with the following constraints:

$$\sum_{a \in A} x_a \mathcal{R}_i(a) \geq \sum_{a \in A} x_a \mathcal{R}_i(a_i'', a_{-i}) \quad \forall i \in I, a_i'' \in A_i \tag{4.24}$$

Note that these linear programs can contain many more variables and constraints than the linear programs for minimax solutions (Section 4.3.1). We now have variables corresponding to each joint action $a \in A$, which can grow exponentially in the number of agents (see also the discussion in Section 5.4.4). For n agents with k actions each, Equation 4.22 specifies k^n constraints, Equation 4.21 specifies nk^2 constraints, and Equation 4.24 specifies nk constraints.

4.7 Conceptual Limitations of Equilibrium Solutions

While equilibrium solutions, in particular the Nash equilibrium, have been adopted as the standard solution concept in MARL, they are not without certain shortcomings. Besides the practical issues noted in Section 4.5, we list some of the most important conceptual limitations in equilibrium solutions:

Sub-optimality In general, finding equilibrium solutions is not synonymous with maximizing expected returns. The only thing we know about a given equilibrium solution is that each agent's policy is a best response to the policies of the other agents, but this does not mean that the agent's expected returns are the best they could be. A simple example of this can be seen in the Prisoner's Dilemma game (Figure 3.2(c)), in which the only Nash equilibrium is the joint action (D,D) giving each agent an expected return of -3, while the joint action (C,C) gives a higher expected return of -1 to each agent but is not a Nash equilibrium (each agent can deviate to improve its return). Similarly, in the Chicken game (Section 4.6), the shown correlated equilibrium π_c achieves an expected return of 5 for each agent, while the joint action (L,L) achieves expected returns of 6 for each agent but is neither a Nash equilibrium nor a correlated equilibrium.

Non-uniqueness Equilibrium solutions may not be unique, which means that there may exist multiple, even infinitely many, equilibria. Each of these equilibria may entail different expected returns for different agents, as can be seen in the three Nash equilibria in the Chicken game. This leads to a difficult challenge: Which of these different equilibria should the agents adopt, and how can they agree on a specific equilibrium? This challenge is known as the "equilibrium selection" problem and has been widely studied

in game theory and economics. In MARL, equilibrium selection can pose a serious challenge for agents that learn concurrently from local observations, which we will discuss further in Section 5.4.2. One approach to tackle equilibrium selection is to use additional criteria such as Pareto optimality (Section 4.8) and social welfare and fairness (Section 4.9) to differentiate between different equilibria.

Incompleteness For games with sequential moves, an equilibrium solution π is incomplete in that it does not specify equilibrium behaviors for *off-equilibrium paths*. An off-equilibrium path is any full history $\hat{h}$ which has probability $\Pr(\hat{h} \mid \pi) = 0$ (Equation 4.3) under the equilibrium π. For example, this could occur due to some temporary disturbance in the agents' executed policies leading to actions not normally prescribed by the policies. In such cases, π does not prescribe actions to bring the interaction back to an on-equilibrium path, meaning a full history $\hat{h}$ with $\Pr(\hat{h} \mid \pi) > 0$ under the equilibrium π. To address such incompleteness, game-theorists have developed refinement concepts such as subgame perfect equilibrium and trembling-hand perfect equilibrium (Selten 1988; Owen 2013).

4.8 Pareto Optimality

As we saw in the previous sections, an equilibrium solution in which agents use mutual best-response policies may be of limited value, because different equilibria may entail very different expected returns for different agents (e.g., Chicken game; see Section 4.7). Moreover, under certain conditions, any feasible and enforceable expected returns can be realized by an equilibrium solution, making the space of equilibrium solutions very large or infinite (folk theorems; see Section 4.4). Therefore, we may want to narrow down the space of equilibrium solutions by requiring additional criteria that a solution must achieve. One such criterion is *Pareto optimality*,[11] defined next:

Definition 9 (Pareto domination and optimality) *A joint policy π is* Pareto-dominated *by another joint policy π' if*

$$\forall i : U_i(\pi') \geq U_i(\pi) \;\; \textit{and} \;\; \exists i : U_i(\pi') > U_i(\pi). \tag{4.25}$$

A joint policy π is Pareto-optimal[12] *if it is not Pareto-dominated by any other joint policy. We then also refer to the expected returns of π as Pareto-optimal.*

11. Named after Italian economist Vilfredo Pareto (1848–1923).

12. Some of the game theory literature uses the terms "Pareto-efficient/inefficient" for Pareto-optimal/dominated, respectively.

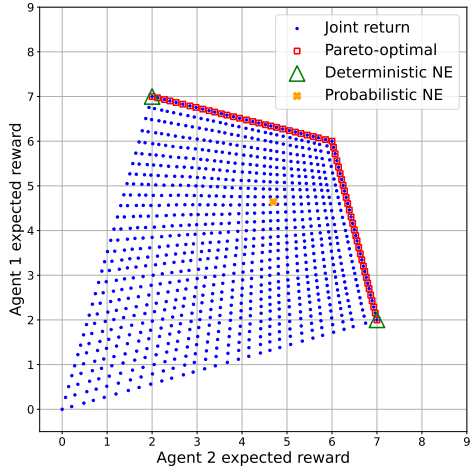


Figure 4.4: Feasible expected joint rewards and Pareto frontier in the Chicken matrix game. Each blue dot shows the expected joint reward obtained by a joint policy. Red squares show joint rewards of Pareto-optimal joint policies. Also shown are the joint rewards corresponding to the deterministic and probabilistic Nash equilibria of the non-repeated game.

Intuitively, a joint policy is Pareto-optimal if there is no other joint policy that improves the expected return for at least one agent, without reducing the expected return for any other agent. In other words, no agent can be better off without making other agents worse off. Every game must have at least one joint policy that is Pareto-optimal. In common-reward games, all Pareto-optimal joint policies achieve the same expected return, which by definition is the maximum possible expected return that any joint policy can achieve in the game.

We illustrate Pareto optimality using the non-repeated Chicken matrix game from Figure 4.3 (page 73). Figure 4.4 shows the convex hull of feasible expected joint returns in this game. Each dot corresponds to the expected joint return achieved by some joint policy (π_1, π_2). In this figure, we discretized the space of joint policies by stepping each policy from $\pi_i(S) = 0$ to $\pi_i(S) = 1$ using a step size of $\frac{1}{30}$, resulting in 30 different policies for each agent, and a total of $30 \cdot 30 = 900$ joint policies. The corners of the convex hull correspond to the four possible deterministic joint policies (i.e., the four joint actions from the game matrix). The expected joint returns obtained by Pareto-optimal joint policies are marked by the red squares. The figure also shows the expected joint returns corresponding to the deterministic and probabilistic Nash equilibria of the game (see Section 4.6 for details).

For infinitely repeated games and under the average reward,[13] there exists a folk theorem that shows that any expected joint return in this hull that is equal to or larger than the minmax value of the agents (Equation 4.17) can be realized by an equilibrium. In the Chicken game, it can be seen from the game's reward matrix that the minmax value for both agents is 2, and so any expected joint return (U_1, U_2) with $U_1, U_2 \geq 2$ in the hull can be realized by means of an equilibrium. This is where Pareto optimality comes in: we can narrow down the space of desirable equilibria by requiring that the expected joint return achieved by an equilibrium also be Pareto-optimal. The Pareto-optimal joint returns are those that reside on the *Pareto frontier* shown by the red squares. Thus, for any given joint policy π, we can project its corresponding expected joint return into the convex hull and detect Pareto optimality if the joint return lies on the Pareto frontier.

We have presented Pareto optimality as a concept to refine equilibrium solutions. However, note that a joint policy can be Pareto-optimal *without* being an equilibrium solution. Yet, Pareto optimality on its own may not be a very useful solution concept. In particular, all joint policies in zero-sum games are Pareto-optimal by definition. In general-sum games, many joint policies may be Pareto-optimal without being a desirable solution, such as joint policies that are Pareto-optimal but result in large differences between the agents' expected returns. We will discuss this latter case further in Section 4.9.

4.9 Social Welfare and Fairness

Pareto optimality states that there is no other solution in which at least one agent is better off without making other agents worse off. However, it does not make any statements about the total amount of rewards and their distribution among the agents. For example, the Pareto frontier in Figure 4.4 contains solutions with expected joint returns ranging from $(7, 2)$ to $(6, 6)$ to $(2, 7)$. Thus, we may consider concepts of *social welfare and fairness* to further constrain the space of desirable solutions.

The study of social welfare and fairness has a long history in economics, and many criteria and social welfare functions have been proposed (Moulin 2004; Fleurbaey and Maniquet 2011; Sen 2018; Amanatidis et al. 2023). The term welfare usually refers to some notion of totality of the agents' returns, while the term fairness relates to the distribution of returns among agents. In this section, we consider two basic definitions of welfare and fairness:

13. Average returns (or average rewards) correspond to discounted returns with $\gamma \to 1$.

Definition 10 (Welfare and Welfare Optimality) *The (social) welfare of a joint policy π is defined as*

$$W(\pi) = \sum_{i \in I} U_i(\pi). \tag{4.26}$$

A joint policy π is welfare-optimal *if $\pi \in \arg\max_{\pi'} W(\pi')$.*

Definition 11 (Fairness and Fairness Optimality) *The (social) fairness of a joint policy π is defined as*[14]

$$F(\pi) = \prod_{i \in I} U_i(\pi). \tag{4.27}$$

A joint policy π is fairness-optimal *if $\pi \in \arg\max_{\pi'} F(\pi')$.*

A welfare-optimal joint policy maximizes the sum of the agents' expected returns, while a fairness-optimal joint policy maximizes the product of the agents' expected returns. This definition of fairness promotes a type of equity between the agents, in the following sense: if we consider a set of joint policies $\pi \in \Pi$ that achieve equal welfare (sum of returns) $W(\pi)$, then the joint policy $\pi \in \Pi$ with the greatest fairness according to $F(\pi)$ is that which gives equal expected return for each agent, that is, $U_i(\pi) = U_j(\pi)$ for all i, j. For example, three different joint policies in a two-agent game that yield expected joint returns of $(1, 5)$, $(2, 4)$, $(3, 3)$ achieve equal welfare of 6 and a respective fairness of 5, 8, and 9.[15] When applying these definitions of welfare and fairness to the example game in Figure 4.4, it can be seen that the only solution that is both welfare-optimal and fairness-optimal is the joint policy that achieves expected joint return of $(6, 6)$. Thus, in this example, we have narrowed the space of desirable solutions to a single solution. Another example of fairness-optimality, in the Battle of the Sexes matrix game, is shown in Figure 4.5.

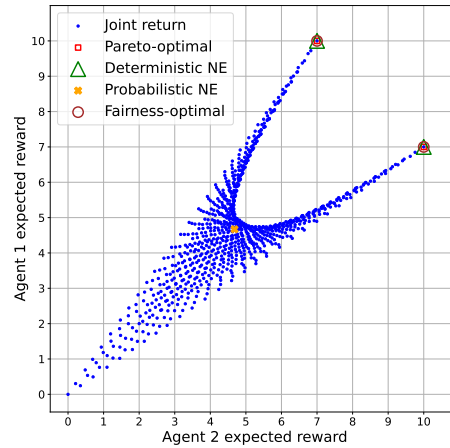
Social welfare and fairness, such as defined here, can be useful in general-sum games but are not so useful in common-reward games and zero-sum games. In common-reward games, in which all agents receive identical reward, welfare and fairness are maximized if and only if the expected return of each agent is maximized; hence, welfare and fairness do not add any useful criteria in this class of games. In two-agent zero-sum games, in which one agent's reward is the negative of the other agent's reward, we know that all minimax solutions

14. This type of fairness is also known as *Nash social welfare*, defined as the geometric mean $\left(\prod_{i \in I} U_i(\pi)\right)^{\frac{1}{n}}$ (Caragiannis et al. 2019; Fan et al. 2023).

15. The careful reader may have noticed some limitations of the simple definition of fairness in Definition 11. For example, if $U_i(\pi) = 0$ for any agent, then it does not matter what returns the other agents obtain under π. And if we allow $U_i(\pi) < 0$, then the expected joint return $(-0.1, 1, 1)$ would be less fair than the expected joint return $(-0.1, -100, 100)$, which is counter-intuitive.

	A	B
A	10,7	2,2
B	0,0	7,10

(a) Battle of the Sexes matrix game.



(b) Joint rewards and fairness-optimal outcomes.

Figure 4.5: Feasible joint rewards and fairness-optimal outcomes in the Battle of the Sexes matrix game. This game models a situation where two people (classically, a man and a woman, hence the name of the game) want to meet in one of two places (A or B) but have different preferences about the best place. Agent 1 prefers place A and agent 2 prefers place B. Joint action (A,B) is preferred over (B,A), since in the latter case both agents end up at their respective least-preferred place. The two deterministic joint policies corresponding to joint actions (A,A) and (B,B) are the only joint policies (in the non-repeated game) that are both Pareto-optimal and fairness-optimal. The figure also shows the only probabilistic Nash equilibrium, which is neither Pareto-optimal nor fairness-optimal.

π have the same unique value $U_i(\pi) = -U_j(\pi)$ for agents i, j. Therefore, all minimax solutions achieve equal welfare and fairness, respectively (besides, the welfare as defined here will always be zero).

It is easy to prove that welfare optimality implies Pareto optimality. To see this, suppose a joint policy π is welfare-optimal but not Pareto-optimal. Since π is not Pareto-optimal, there exists a joint policy π' such that $\forall i : U_i(\pi') \geq U_i(\pi)$ and $\exists i : U_i(\pi') > U_i(\pi)$. However, it follows that $\sum_i U_i(\pi') > \sum_i U_i(\pi)$ and, therefore, π cannot be welfare-optimal (contradiction). Thus, π must also be Pareto-optimal if it is welfare-optimal. Note that Pareto optimality does not in general imply welfare optimality, hence welfare optimality is a stronger requirement. Moreover, fairness optimality does not imply Pareto optimality, and vice versa.

4.10 No-Regret

The equilibrium solution concepts discussed in the previous sections are based on mutual best responses between agents, and are thus a function of the agents' policies. Another category of solution concepts is based on the notion of *regret*, which measures the difference between the rewards an agent received and the rewards it could have received if it had chosen a different action (or policy) in past episodes against the observed actions (or policies) of the other agents in these episodes. An agent is said to have *no-regret* if, in the limit of infinitely many episodes, the agent's average regret across the episodes is at most zero. Therefore, no-regret considers the performance of learning agents across multiple episodes, which is in contrast to the other solution concepts introduced in this chapter that only consider a single joint policy (and not *how* this joint policy was learned). In this sense, no-regret can be viewed as an example of the prescriptive agenda discussed in Section 1.5, which is concerned with the performance of agents during learning.

There are multiple ways in which regret can be defined. We will first give a standard definition of regret for non-repeated normal-form games, which is based on comparing the rewards of different actions in the episodes. This definition will then be extended to sequential-move games. Let a^e denote the joint action from episodes $e = 1, ..., z$. Agent i's regret for not having chosen the best single action across these episodes is defined as

$$Regret_i^z = \max_{a_i \in A_i} \sum_{e=1}^{z} \left[\mathcal{R}_i(\langle a_i, a^e_{-i} \rangle) - \mathcal{R}_i(a^e) \right]. \tag{4.28}$$

An agent is said to have no-regret if its average regret in the limit of $z \to \infty$ is at most zero. As a solution concept, no-regret requires that all agents in the game have no-regret.

Definition 12 (No-regret) *In a general-sum game with n agents, the agents have* no-regret *if*

$$\forall i : \lim_{z \to \infty} \frac{1}{z} Regret_i^z \leq 0. \tag{4.29}$$

Similar to ϵ-Nash equilibrium (Section 4.5), we may replace the ≤ 0 in Equation 4.29 with $\leq \epsilon$, for $\epsilon > 0$, to obtain an ϵ-no-regret.

As a concrete example, Figure 4.6 shows ten episodes of two agents in the non-repeated Prisoner's Dilemma matrix game. After the episodes, agent 1 has received a total reward of -21. Always choosing C in the episodes (against the observed actions of agent 2) would have resulted in a total reward of -30, while always choosing D would have resulted in a total reward of -15. Thus,

Episode e	1	2	3	4	5	6	7	8	9	10
Action a_1^e	C	C	D	C	D	D	C	D	D	D
Action a_2^e	C	D	C	D	D	D	C	C	D	C
Reward $\mathcal{R}_1(a^e)$	-1	-5	0	-5	-3	-3	-1	0	-3	0
Reward $\mathcal{R}_1(\langle C, a_2^e \rangle)$	-1	-5	-1	-5	-5	-5	-1	-1	-5	-1
Reward $\mathcal{R}_1(\langle D, a_2^e \rangle)$	0	-3	0	-3	-3	-3	0	0	-3	0

Figure 4.6: Ten episodes between two agents in the non-repeated Prisoner's Dilemma matrix game. The bottom two rows show agent 1's rewards for always choosing actions C/D against agent 2's observed actions in the episodes.

action D was the "best" action against the observed actions of agent 2, and so agent 1's regret is $Regret_1^{10} = -15 + 21 = 6$ with an average regret (dividing by 10) of 0.6. Indeed, in Prisoner's Dilemma, D is a dominant action in that it is a best response against both D and C (see the reward matrix in Figure 3.2(c) (page 46) to verify this). In order for agent 1 to achieve no-regret, the episodes would need to continue in such a way that agent 1's average regret across the episodes goes to zero.

We can generalize the definition of no-regret to stochastic games and POSGs, by redefining the regret over policies rather than actions. For each agent $i \in I$, let Π_i be a finite space of policies that agent i can select from. Let π^e denote the joint policy from episodes $e = 1, ..., z$, with $\pi_i^e \in \Pi_i$ for all $i \in I$. Then, agent i's regret for not having chosen the best policy across these episodes is defined as

$$Regret_i^z = \max_{\pi_i \in \Pi_i} \sum_{e=1}^{z} \left[U_i(\langle \pi_i, \pi_{-i}^e \rangle) - U_i(\pi^e) \right]. \tag{4.30}$$

With this definition of regret, Definition 12 applies to all of the game models introduced in Chapter 3. (For non-repeated normal-form games, Equation 4.30 is equivalent to Equation 4.28 if we define each Π_i to be the set of deterministic policies corresponding to each action $a_i \in A_i$.)

Our previous example in Prisoner's Dilemma illustrates an important conceptual limitation of regret, which is that it assumes that the actions or policies of other agents $-i$ remain fixed in the episodes. This assumption is sensible if the other agents use constant policies that do not change between episodes. However, if the other agents adapt their policies based on past episodes, then this assumption is of course violated. Therefore, regret does not actually quantify what would have happened under counterfactual situations. The second limitation, which is a result of the first limitation, is that minimizing regret is

not necessarily equivalent to maximizing returns (Crandall 2014). For example, in non-repeated and finitely repeated Prisoner's Dilemma, the only joint policy that has no-regret is for both agents to always choose D. This is analogous to the fact that mutual best-response policies do not necessarily entail maximum returns for the agents, as we discussed in Section 4.7.

Various alternative definitions of regret exist (e.g., de Farias and Megiddo 2003; Lehrer 2003; Chang 2007; Zinkevich et al. 2007; Arora, Dekel, and Tewari 2012; Crandall 2014). For example, for normal-form games in which agents can choose from more than two actions, rather than replacing all of agent i's past actions as in Equation 4.28, we may replace only the specific occurrences of a given action a_i' in the history with a different action a_i. This latter definition of regret is also known as *conditional* (or internal) regret, while the definition in Equation 4.28 is known as *unconditional* (or external) regret. Furthermore, no-regret solutions have connections to equilibrium solutions. In particular, in two-agent zero-sum normal-form games, the empirical distribution of joint actions produced by agents that have no external regret converges to the set of minimax solutions; and in general-sum normal-form games, the empirical distribution of joint actions converges to the set of coarse correlated equilibria if the agents have no external regret, and to the set of correlated equilibria if the agents have no internal regret (Hart and Mas-Colell 2000; Young 2004). (See also Section 5.2 for a discussion of convergence types.) We will revisit these two regret definitions and their connection to correlated equilibrium in Section 6.5.

4.11 The Complexity of Computing Equilibria

Before we turn to MARL algorithms in Chapters 5 and 6 as a method to compute solutions for games, it is instructive to ask: How difficult is it, in terms of computational complexity, to compute an equilibrium solution for a game? Do algorithms exist that can compute equilibria *efficiently*, meaning in polynomial time in the size of the game?

These and related questions are studied in algorithmic game theory, which is a research area at the interface of computer science and game theory. Many complexity results exist for various special types of games, and we recommend the books of Nisan et al. (2007) and Roughgarden (2016) for a broad discussion. Here, we focus on non-repeated normal-form games, which are the building block of the more complex (partially observable) stochastic game models. Thus, we can expect any complexity results for normal-form games to be a lower bound on the complexity for the more complex game models.

Most computer scientists will have some familiarity with the complexity classes P and NP. P includes all decision problems whose solutions (if they exist) can be computed efficiently in polynomial time in the size of the problem instance. NP includes all decision problems whose solutions (if they exist) can be computed in polynomial time by a non-deterministic Turing machine. Unfortunately, these familiar complexity classes are not a good fit for the problem of computing an equilibrium, since P/NP characterize *decision problems* which may or may not have solutions, while we know that games *always* have at least one equilibrium solution. On the other hand, computing an equilibrium that satisfies additional properties *is* a decision problem, since such solutions may or may not exist. Such problems include computing equilibria that:

- are Pareto optimal;
- achieve a certain minimum expected return for each agent;
- achieve a certain minimum social welfare (sum of returns);
- assign zero or positive probability to certain actions of certain agents.

All of these problems are known to be NP-hard (Gilboa and Zemel 1989; Conitzer and Sandholm 2008).

Some types of games and equilibria do admit polynomial-time algorithms. Computing a minimax solution in a two-agent zero-sum non-repeated normal-form game can be formulated via linear programming (Section 4.3.1), which can be solved in polynomial time. Similarly, computing a correlated equilibrium in a general-sum non-repeated normal-form game can be done in polynomial time via a linear program (Section 4.6.1). However, computing a Nash equilibrium in general-sum non-repeated normal-form games — henceforth simply called NASH — cannot be solved via linear programming, due to the independence assumption between policies in a Nash equilibrium.

Problems that always have solutions, such as NASH, are known as *total search problems*. Section 4.11.1 will present one subclass of total search problems, called PPAD. It turns out that NASH is a *complete* problem in PPAD, meaning that any other problem in PPAD can be reduced to NASH. We will discuss the implications for MARL in Section 4.11.2.

4.11.1 PPAD Complexity Class

PPAD (short for "polynomial parity argument for directed graphs") describes a special class of total search problems. We define PPAD by giving one of its

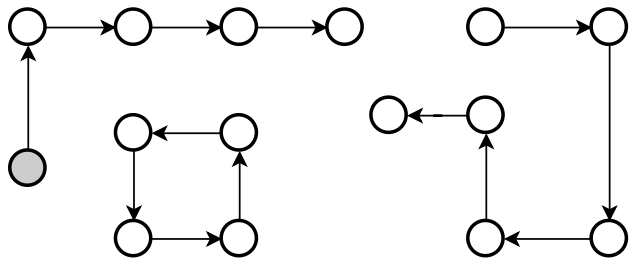


Figure 4.7: Any instance of END-OF-LINE consists of a set of paths and cycles, such as the instance shown here. Given the *Parent* / *Child* functions and a source node (a node with no parent node; here shaded in gray), is there an efficient polynomial-time algorithm to find a sink node (a node with no child node) or a different source node?

complete problems to which all other problems in PPAD can be reduced.[16] This PPAD-complete problem is called END-OF-LINE and is defined as follows:

Definition 13 (END-OF-LINE) *Let $G(k) = (V, E)$ be a directed graph consisting of*

- *a finite set V containing 2^k nodes (each node is represented as a bit-string of length k)*
- *a finite set $E = \{(a, b) \mid a, b \in V\}$ of directed edges (from node a to node b, for $a, b \in V$) such that:*

$$\text{if } (a, b) \in E \text{ then } \nexists a' \neq a : (a', b) \in E \text{ and } \nexists b' \neq b : (a, b') \in E$$

Assume functions Parent(v) and Child(v) that, respectively, return the parent node (if any) and child node (if any) of node $v \in V$. These functions are represented as boolean circuits with k input bits and k output bits, and run in time polynomial in k. Given access to functions Parent and Child (but not E), and a node $s \in V$ such that Parent$(s) = \emptyset$, find a node $e \neq s$ such that either Child$(e) = \emptyset$ or Parent$(e) = \emptyset$.[17]

Figure 4.7 shows an illustration of a END-OF-LINE problem instance. The restriction on E in Definition 13 means that any node in the graph can have at most one parent node and at most one child node. A node with no parent is

16. Problem A can be "reduced" to problem B if there exist polynomial-time algorithms to transform any instance of A into an equivalent instance of B, and any solution of the instance of B to a solution of the original instance of A.

17. To indicate that a node has no parent/child, the respective circuit function simply outputs the same input node.

also called a source node, while a node with no child is called a sink node. The "parity argument" in PPAD refers to the fact that any source node in this graph always has a corresponding sink node. Therefore, if a source node s is given, then we know that a node e must exist. The node e can, in principle, be found by following the directed path starting at the given source node s. However, since we are only given the functions *Parent* and *Child* (and not E), the only obvious way to follow this path is by repeatedly calling the function *Child* starting at the source node s. Therefore, finding a sink node may require exponential time in the worst case, since there are 2^k nodes.

Why should we care about PPAD? Just as it is unknown whether efficient polynomial-time algorithms exist to solve NP-complete problems (the big "P = NP?" question), it is also unknown whether efficient algorithms exist for PPAD-complete problems ("P = PPAD?"). Indeed, PPAD contains problems for which researchers have tried for decades to find efficient algorithms, including the classic Brouwer fixed-point problem and finding Arrow-Debreu equilibria in markets (see Papadimitriou (1994) for a more complete list). PPAD has also been shown to be hard under cryptographic assumptions (Bitansky, Paneth, and Rosen 2015; Garg, Pandey, and Srinivasan 2016; Choudhuri et al. 2019). There are currently no known algorithms that can efficiently solve END-OF-LINE (i.e., in time polynomial in k) and, thus, any other problem in PPAD. Therefore, establishing that a problem is PPAD-complete is a good indicator that no efficient algorithms exist to solve the problem.

4.11.2 Computing ϵ-Nash Equilibrium Is PPAD-Complete

We return to our initial question, "Do algorithms exist that can compute equilibria *efficiently*, in polynomial time in the size of the game?" Unfortunately, the answer is very likely negative. It has been proven that NASH is PPAD-complete, at first for games with three or more agents (Daskalakis, Goldberg, and Papadimitriou 2006, 2009), and shortly after even for games with two agents (Chen and Deng 2006). This means that finding a Nash equilibrium in a non-repeated normal-form game can be described as finding an e-node in an equivalent END-OF-LINE instance. The completeness also means that any other problem in PPAD, including Brouwer fixed-points, can be reduced to NASH.

More precisely, the PPAD-completeness of NASH was proven for approximate ϵ-Nash equilibrium (Section 4.5) for certain bounds on $\epsilon > 0$; and for

exact equilibria (when $\epsilon = 0$) in games with two agents.[18] The use of ϵ-Nash equilibrium is to account for the fact that a Nash equilibrium may involve irrational-valued probabilities in games with more than two agents. Therefore, the PPAD-completeness of NASH also includes approximation schemes for computing Nash equilibria, such as MARL algorithms, which may only learn approximate solutions given a finite number of interactions in the game.

The implication for us is that MARL is unlikely to be a magic bullet for solving games: since no efficient algorithms are known to exist for PPAD-complete problems, it is unlikely that efficient MARL algorithms exist to compute Nash equilibria in polynomial time. Much of the research in MARL has focused on identifying and exploiting structures (or assumptions) in certain game types that lead to improved performance. However, the PPAD-completeness of NASH tells us that, in general and without such assumed structures, it is likely that *any* MARL algorithm still requires exponential time in the worst case.

4.12 Summary

This chapter introduced a range of solution concepts for games, to define optimal policies for agents in the game. The main concepts are the following:

- A *solution* for a game is a joint policy (typically including one policy for each agent) that satisfies certain conditions which are expressed in terms of the expected returns for agents and the relationship between the returns.
- Many solution concepts can be compactly represented based on the notion of *best responses*. A best-response policy maximizes the expected returns of one agent against a set of given policies for the other agents.
- There exists a series of increasingly general equilibrium solution concepts, including *minimax* for two-agent zero-sum games, and *Nash equilibrium* and *(coarse) correlated equilibrium* for general-sum games with two or more agents. These equilibrium solutions are all based on the idea that every agent is best-responding to all other agents under the equilibrium, and hence no agent can unilaterally deviate from the equilibrium to increase its returns.
- Games may have a unique equilibrium solution, or they may have multiple (even infinitely many) equilibria that can yield different expected returns for

18. As discussed in Section 4.5, an ϵ-Nash equilibrium may not be close to any actual Nash equilibrium, even if the Nash equilibrium is unique. The problem of approximating an actual Nash equilibrium within a desired distance, as measured in the policy space via norms such as L1 and L2, is in fact much harder than computing an ϵ-Nash equilibrium. This latter problem, for general-sum normal-form games with three or more agents, is complete for a different complexity class called FIXP (Etessami and Yannakakis 2010).

the agents. Thus, learning an equilibrium solutions is not necessarily the same as maximizing the expected returns for all agents.

- Different refinement concepts exist that can be combined with equilibrium solutions to narrow the space of desirable solutions, such as *Pareto optimality* and concepts based on *social welfare and fairness*. For example, we may seek a Nash equilibrium that is also Pareto-optimal.
- *No-regret* is an alternative solution concept that considers the alternative returns an agent could have received if it had used different policies in past episodes of the game against the observed policies of the other agents. A joint policy achieves no-regret if the average regret for each agent goes to zero in the limit of infinitely many episodes.
- Computing a Nash equilibrium in general-sum normal-form games is a complete problem for the *PPAD complexity class*. There are currently no known efficient (i.e., polynomial time) algorithms to solve PPAD-complete problems. The implication is that there probably do not exist efficient MARL algorithms that learn Nash equilibria for general games.

Now that we are equipped with different solution concepts for the game models introduced in Chapter 3, the next two chapters in this part of the book will describe different families of MARL algorithms that are designed to learn such solutions under certain conditions. Chapter 5 will begin by outlining a general learning process and different convergence types with which MARL algorithms can learn or approximate solutions, as well as the key challenges when trying to learn game solutions using MARL methods.